V3 GRAND MADURAI

Developer Scaffolding Guide

Code-Adjacent Patterns, Types, and Skeletons for Engineering

Monorepo · Domain Types · Engine Skeletons · API Routes · Workflows · DB Schema · UI Contracts

March 2026 | Companion to Blueprint v2 + Implementation Plan

CONFIDENTIAL

1. Monorepo Bootstrap

This section specifies the pnpm workspace layout, per-package configuration patterns, and shared tooling. A senior engineer should be able to initialize the full monorepo from these specs in under two hours.

1.1 Workspace Root

Package manager: pnpm 9+ with workspaces. Turborepo for task orchestration (build, test, lint run in dependency-aware parallel). Node 20 LTS.

pnpm-workspace.yaml

```
packages:
  - "packages/*"
```

Root package.json (key fields only)

```
{
  "name": "v3grand",
  "private": true,
  "packageManager": "pnpm@9.12.0",
  "engines": { "node": ">=20.0.0" },
  "scripts": {
    "build":     "turbo run build",
    "test":      "turbo run test",
    "test:ci":   "turbo run test -- --reporter=junit",
    "lint":      "turbo run lint",
    "typecheck": "turbo run typecheck",
    "format":    "prettier --write .",
    "db:migrate": "pnpm --filter @v3grand/db migrate",
    "db:seed":    "pnpm --filter @v3grand/db seed",
    "dev":       "turbo run dev --parallel"
  }
}
```

turbo.json

```
{
  "$schema": "https://turbo.build/schema.json",
  "tasks": {
    "build":     { "dependsOn": ["^build"], "outputs": ["dist/**"] },
    "test":      { "dependsOn": ["build"] },
    "lint":      {},
    "typecheck": { "dependsOn": ["^build"] },
    "dev":       { "cache": false, "persistent": true }
  }
}
```

Root tsconfig.json (project references base)

```
{
  "compilerOptions": {
    "target": "ES2022", "module": "NodeNext",
    "moduleResolution": "NodeNext", "strict": true,
    "esModuleInterop": true, "skipLibCheck": true,
    "declaration": true, "declarationMap": true,
    "sourceMap": true, "outDir": "dist",
    "rootDir": "src", "composite": true
  }
}
```

1.2 Package Layout

Package	npm Name	Purpose	Key Dependencies
packages/core	@v3grand/core	Domain types, event schemas, DTOs, shared utils, Zod validators	zod
packages/engines	@v3grand/engines	Pure computation: Factor, Underwriter, MC, Budget, Decision, SCurve	@v3grand/core
packages/db	@v3grand/db	Drizzle ORM schema, migrations, seed, query helpers	@v3grand/core, drizzle-orm, drizzle-kit, pg
packages/api	@v3grand/api	Fastify server: routes, middleware, RBAC, WebSocket	@v3grand/core, engines, db, fastify, @fastify/websocket, zod
packages/workflows	@v3grand/workflows	Temporal workflow + activity defs	@v3grand/core, engines, db, @temporalio/workflow, @temporalio/activity
packages/ui	@v3grand/ui	Next.js 14 app: pages, components, hooks	@v3grand/core (types only), next, react, @tanstack/react-query, recharts, zustand
packages/infra	@v3grand/infra	Docker Compose, Dockerfiles, deploy scripts, Terraform stubs	N/A (no TS build)

1.3 Per-Package Config Pattern

Every package under packages/ follows the same structure:

```
packages/<name>/
  src/
    index.ts          # barrel export
  tsconfig.json       # extends root, sets rootDir/outDir
  package.json        # scoped name, scripts, deps
  vitest.config.ts    # per-package test config (if applicable)
```

Per-package tsconfig.json (example: packages/engines)

```
{
  "extends": "../../tsconfig.json",
  "compilerOptions": {
    "rootDir": "src", "outDir": "dist"
  },
  "include": ["src"],
  "references": [
    { "path": "../core" }
  ]
}
```

Per-package package.json (example: packages/engines)

```
{
  "name": "@v3grand/engines",
  "version": "0.1.0",
  "main": "dist/index.js",
  "types": "dist/index.d.ts",
  "scripts": {
    "build": "tsc -b",
    "test": "vitest run",
    "typecheck": "tsc --noEmit",
    "lint": "eslint src/"
  },
```

```
  "dependencies": {
    "@v3grand/core": "workspace:*"
  }
}
```

1.4 Shared Tooling

ESLint (.eslintrc.cjs at root)

```
module.exports = {
  root: true,
  parser: "@typescript-eslint/parser",
  extends: [
    "eslint:recommended",
    "plugin:@typescript-eslint/strict-type-checked",
  ],
  rules: {
    "@typescript-eslint/no-floating-promises": "error",
    "@typescript-eslint/no-unsafe-assignment": "error",
    "no-console": ["warn", { allow: ["warn", "error"] }],
  },
  ignorePatterns: ["dist/", "node_modules/"],
};
```

Prettier (.prettierrc)

```
{ "semi": true, "singleQuote": true, "trailingComma": "all",
  "printWidth": 100, "tabWidth": 2 }
```

Vitest (root vitest.config.ts)

```
import { defineConfig } from "vitest/config";
export default defineConfig({
  test: {
    globals: true,
    environment: "node",
    include: ["packages/*/src/**/*.test.ts"],
    coverage: { provider: "v8", reporter: ["text", "lcov"],
                thresholds: { lines: 80, functions: 80 } },
  },
});
```

1.5 CI Pipeline (GitHub Actions)

Single workflow file .github/workflows/ci.yml. Three jobs:

Job	Trigger	Steps	Key Config
check	push + PR	pnpm install → turbo typecheck → turbo lint → turbo test:ci	Runs on ubuntu-latest, node 20. Turborepo remote cache via Vercel.
build	push to main	turbo build → docker build per service	Builds API, workflows, UI containers. Pushes to ECR/GHCR.
deploy-staging	push to main (after build)	Pull images → deploy to staging env	Only if check+build pass. Manual gate for production.

Note: infra/ is not built by Turbo. It contains only Docker Compose and Terraform files.

1.6 Package Dependency Rules (Enforced by ESLint)

To prevent architecture erosion, use eslint-plugin-import with custom boundaries:

Package	May Import From	Must Never Import From
core	(nothing — leaf)	engines, api, db, workflows, ui
engines	core	api, db, workflows, ui
db	core	engines, api, workflows, ui
workflows	core, engines, db	api, ui
api	core, engines, db, workflows	ui
ui	core (types only)	engines, db, api, workflows

The critical invariant: engines/ has zero I/O, zero framework dependencies. It receives plain objects and returns plain objects. This makes every engine independently testable, extractable to a microservice, and reusable across asset classes.

2. Domain Types & DTOs (packages/core)

All types below live in packages/core/src/. They are pure TypeScript interfaces and Zod schemas — no logic, no I/O. Every other package imports from @v3grand/core.

2.1 File Layout

```
packages/core/src/
  index.ts               # barrel re-export
  types/
    deal.ts              # Deal, Property, Partnership
    assumptions.ts       # Market, Financial, Opex
    capex.ts             # CapexPlan, CapexLineItem
    scenario.ts          # Scenario, ScenarioSet
    construction.ts      # BudgetLine, RFI, ChangeOrder
    risk.ts              # Risk, RiskRegister
    recommendation.ts    # RecommendationState, Gate
    engine-results.ts    # EngineResults (versioned wrapper)
    audit.ts             # AuditLogEntry
    lifecycle.ts         # LifecyclePhase, Milestone, Task
  dto/
    engine-io.ts         # all engine input/output DTOs
    views.ts             # DealDashboardView, ScenarioExplorerView, BudgetTrackerView
  events/
    event-types.ts       # discriminated union of all domain events
  schemas/
    validators.ts        # Zod schemas mirroring each type
```

2.2 Core Entity Interfaces

Deal (types/deal.ts)

```
export interface Deal {
  id: string;                      // UUID v7 (time-sortable)
  name: string;                    // "V3 Grand Madurai"
  assetClass: "hotel";             // extensible later: "mixed-use" | "commercial"
  status: DealStatus;              // "draft" | "active" | "on-hold" | "closed"
  lifecyclePhase: LifecyclePhaseId; // "pre-development" | "construction" | "pre-opening"
  currentMonth: number;            // months since deal inception (0-indexed)
  version: number;                 // optimistic concurrency control
  property: Property;
  partnership: Partnership;
  marketAssumptions: MarketAssumptions;
  financialAssumptions: FinancialAssumptions;
  capexPlan: CapexPlan;
  opexModel: OpexModel;
  scenarios: ScenarioSet;
  createdAt: string;               // ISO 8601
  updatedAt: string;
}
```

Property

```
export interface Property {
  location: { city: string; state: string; country: string;
              lat: number; lng: number; distanceToAirportKm: number; };
  landArea: { sqft: number; acres: number; };
  grossBUA: { phase1Sqft: number; phase2Sqft: number; totalSqft: number; };
  keys: { phase1: number; phase2: number; total: number; };
  roomTypes: RoomType[];           // { name, count, sqft, adrPremium }
```

```
  amenities: string[];             // ["restaurant", "pool", "banquet-hall"]
  starRating: 3 | 4 | 5;
}
```

Partnership

```
export interface Partnership {
  structure: "jv" | "sole" | "syndication";
  partners: Partner[];
}
export interface Partner {
  id: string; name: string; equityPct: number;  // 0-1
  role: "lead-investor" | "co-investor" | "operator" | "lender";
  commitmentCr: number;            // in Crores
}
```

MarketAssumptions (types/assumptions.ts)

```
export interface MarketAssumptions {
  segments: MarketSegment[];       // { name, pctMix, adrPremium, seasonality[] }
  occupancyRamp: number[];         // year 0..9, e.g. [0, 0.35, 0.55, 0.68, 0.75, 0.78, ...]
  adrBase: number;                 // INR, year-1 blended ADR
  adrStabilized: number;           // INR, stabilized ADR
  adrGrowthRate: number;           // annual growth, e.g. 0.05
  revenueMix: { rooms: number; fb: number; banquet: number; other: number; };
  seasonality: number[];           // 12 monthly multipliers summing to 12
  compSet: CompSetEntry[];         // { name, keys, adr, occ, revpar }
}
```

FinancialAssumptions

```
export interface FinancialAssumptions {
  wacc: number;                    // e.g. 0.138
  riskFreeRate: number;            // e.g. 0.072
  equityRatio: number;             // e.g. 0.50
  debtRatio: number;
  debtInterestRate: number;        // e.g. 0.10
  debtTenorYears: number;          // e.g. 12
  exitCapRate: number;             // e.g. 0.08
  exitMultiple: number;            // e.g. 12
  taxRate: number;                 // e.g. 0.252
  inflationRate: number;           // e.g. 0.05
  managementFeePct: number;        // e.g. 0.03
  incentiveFeePct: number;         // e.g. 0.08
  ffAndEReservePct: number;        // e.g. 0.04
  workingCapitalDays: number;      // e.g. 30
  targetIRR: number;
  targetEquityMultiple: number;
  targetDSCR: number;
}
```

CapexPlan & CapexLineItem (types/capex.ts)

```
export interface CapexPlan {
  phase1: CapexPhase;              // total: 35.7 Cr
  phase2: CapexPhase;              // total: 7.3 Cr  (conditional)
  contingencyPct: number;
}
export interface CapexPhase {
  totalBudgetCr: number;
  items: CapexLineItem[];
}
export interface CapexLineItem {
```

```
  id: string;
  costCode: string;                // e.g. "STRUC-001"
  description: string;
  category: "land" | "structure" | "mep" | "interiors" | "ffne" | "pre-opening" |
"contingency";
  budgetAmount: number;            // INR
  committedAmount: number;
  spentAmount: number;
  forecastAmount: number;          // current best estimate
  startMonth: number;              // 0-indexed from deal start
  endMonth: number;
  curveType: "s-curve" | "linear" | "front-loaded" | "back-loaded";
}
```

OpexModel

```
export interface OpexModel {
  departments: USALIDepartment[];
  undistributed: USALIDepartment[];
  fixedCharges: FixedCharge[];
}
export interface USALIDepartment {
  name: string;                    // "Rooms", "F&B", "Banquets", "Other"
  costPctOfRevenue: number;        // e.g. 0.28
  fixedVarSplit: { fixed: number; variable: number; };
  fixedFloorMonthly: number;       // INR, minimum monthly cost
  gmSavingsPct: number;            // GM negotiated savings
}
```

Scenario & ScenarioSet (types/scenario.ts)

```
export interface Scenario {
  id: string;
  name: "bear" | "base" | "bull";
  probability: number;             // 0-1, must sum to 1
  occupancyStabilized: number;
  adrStabilized: number;           // INR
  ebitdaMargin: number;
  mouRealizationPct: number;       // MOU conversion rate
  phase2Trigger: boolean;          // does Phase 2 proceed?
}
export interface ScenarioSet {
  bear: Scenario;
  base: Scenario;
  bull: Scenario;
}
```

Construction: BudgetLine, RFI, ChangeOrder (types/construction.ts)

```
export interface BudgetLine {
  id: string;
  costCode: string;                // foreign key to CapexLineItem.costCode
  month: number;
  planned: number;                 // INR
  committed: number;
  actual: number;
  forecast: number;
  varianceToPlanned: number;       // actual - planned
  varianceToCurrent: number;       // actual - forecast
}

export interface RFI {
```

```
  id: string;
  dealId: string;
  number: number;                  // sequential per deal
  title: string;
  description: string;
  priority: "low" | "medium" | "high" | "critical";
  status: "open" | "responded" | "closed";
  assigneeId: string;
  costImpact: number | null;       // INR, null if unknown
  scheduleImpactDays: number | null;
  createdAt: string;
  resolvedAt: string | null;
}

export interface ChangeOrder {
  id: string;
  dealId: string;
  number: number;
  title: string;
  description: string;
  costImpact: number;              // INR, positive = increase
  scheduleImpactDays: number;
  status: "draft" | "submitted" | "under-review" | "approved" | "rejected";
  approverRole: "lead-investor" | "operator";
  submittedBy: string;
  approvedBy: string | null;
  affectedCostCodes: string[];
  createdAt: string;
  decidedAt: string | null;
}
```

Risk (types/risk.ts)

```
export interface Risk {
  id: string;
  dealId: string;
  category: "market" | "construction" | "financial" | "regulatory" | "operational";
  title: string;
  description: string;
  probability: 1 | 2 | 3 | 4 | 5; // 1=rare, 5=almost certain
  impact: 1 | 2 | 3 | 4 | 5;      // 1=negligible, 5=catastrophic
  riskScore: number;               // probability * impact
  status: "identified" | "mitigating" | "accepted" | "closed";
  mitigation: string;
  costExposureCr: number;
  owner: string;                   // userId
}
```

RecommendationState (types/recommendation.ts)

```
export type RecommendationVerdict =
  | "INVEST" | "HOLD" | "DE-RISK" | "EXIT" | "DO-NOT-PROCEED";

export interface RecommendationState {
  id: string;
  dealId: string;
  version: number;                 // monotonically increasing
  timestamp: string;
  verdict: RecommendationVerdict;
  confidence: number;              // 0-100
  triggerEvent: string;            // event that caused this evaluation
```

```
  factorSnapshot: FactorScoreOutput;
  proformaSnapshot: { irr: number; npv: number; equityMultiple: number; dscr: number; };
  mcSnapshot: { p10Irr: number; p50Irr: number; p90Irr: number; probNpvNeg: number; };
  gateResults: GateCheck[];
  explanation: string;             // human-readable why
  previousVerdict: RecommendationVerdict | null;
  isFlip: boolean;                 // verdict differs from previous
}

export interface GateCheck {
  name: string;                    // e.g. "IRR > WACC + 200bps"
  passed: boolean;
  actual: number;
  threshold: number;
}
```

EngineResults & AuditLog

```
export interface EngineResult {
  id: string;
  dealId: string;
  engineName: "factor" | "underwriter" | "montecarlo" | "budget" | "decision" | "scurve";
  version: number;
  input: Record<string, unknown>;  // JSONB, engine-specific
  output: Record<string, unknown>; // JSONB, engine-specific
  durationMs: number;
  triggeredBy: string;             // event name
  createdAt: string;
}

export interface AuditLogEntry {
  id: string;
  timestamp: string;
  dealId: string;
  userId: string;
  role: string;
  module: string;                  // "underwriter" | "construction" | "decision" | ...
  action: string;                  // "assumption.updated" | "change-order.approved" | ...
  entityType: string;
  entityId: string;
  diff: Record<string, unknown>;   // { field: { old, new } }
}
```

2.3 Engine I/O DTOs (dto/engine-io.ts)

Each engine has an Input and Output type. These are the exact shapes passed to and returned from engine functions.

Factor Engine

```
export interface FactorScoreInput {
  deal: Deal;                      // full deal object
  macroIndicators?: {              // optional external data
    repoRate: number; cpi: number; hotelRevParIndex: number;
    stateGdpGrowth: number; airTrafficGrowth: number;
  };
}
export interface FactorScoreOutput {
  compositeScore: number;          // 1.0 - 5.0
  domainScores: {
    global: { score: number; weight: 0.20; factors: FactorDetail[]; };
```

```
    local:  { score: number; weight: 0.25; factors: FactorDetail[]; };
    asset:  { score: number; weight: 0.35; factors: FactorDetail[]; };
    sponsor:{ score: number; weight: 0.20; factors: FactorDetail[]; };
  };
  impliedDiscountRate: number;     // Rf + 500bps + (5-composite)*200bps
  impliedCapRate: number;
}
export interface FactorDetail {
  name: string; score: number; weight: number; rationale: string;
}
```

Underwriter / ProForma

```
export interface ProFormaInput {
  deal: Deal;
  scenarioKey: "bear" | "base" | "bull";
  overrides?: Partial<FinancialAssumptions>;
}
export interface ProFormaOutput {
  scenarioKey: string;
  years: YearProjection[];         // 10 years
  totalInvestment: number;
  equityInvestment: number;
  debtDrawdown: number;
  cashFlows: number[];             // 11 entries: year 0 (negative) + years 1-10
  irr: number;
  npv: number;
  equityMultiple: number;
  paybackYear: number;
  avgDSCR: number;
  exitValue: number;
}
export interface YearProjection {
  year: number;
  occupancy: number; adr: number; revpar: number;
  roomRevenue: number; totalRevenue: number;
  departmentalProfit: number;
  undistributedExpenses: number;
  gop: number; gopMargin: number;
  ebitda: number; ebitdaMargin: number;
  debtService: number; fcfe: number;
}
```

Monte Carlo

```
export interface MCInput {
  deal: Deal;
  iterations: number;              // default 5000
  seed?: number;                   // for reproducibility
}
export interface MCOutput {
  iterations: number;
  irrDistribution: { p5: number; p10: number; p25: number;
    p50: number; p75: number; p90: number; p95: number; };
  npvDistribution: { p5: number; p10: number; p25: number;
    p50: number; p75: number; p90: number; p95: number; };
  probNpvNegative: number;         // 0-1
  probIrrBelowWacc: number;
  probPhase2Trigger: number;
  sensitivityRanking: { factor: string; elasticity: number; }[];
  histogram: { bucket: number; count: number; }[]; // for UI chart
}
```

Budget Variance

```
export interface BudgetAnalysisInput {
  deal: Deal;
  asOfMonth: number;
}
export interface BudgetAnalysisOutput {
  asOfMonth: number;
  totalBudget: number;
  totalCommitted: number;
  totalSpent: number;
  totalForecast: number;
  varianceToOriginal: number;
  varianceToCurrent: number;
  byCategory: CategoryVariance[];
  byCostCode: CostCodeVariance[];
  sCurveData: { month: number; planned: number; actual: number; forecast: number; }[];
  alerts: BudgetAlert[];
}
```

Decision Engine

```
export interface DecisionInput {
  deal: Deal;
  factorResult: FactorScoreOutput;
  proformaResults: Record<string, ProFormaOutput>;  // keyed by scenario
  mcResult: MCOutput;
  budgetResult?: BudgetAnalysisOutput;
  currentRecommendation?: RecommendationState;
}
export interface DecisionOutput {
  verdict: RecommendationVerdict;
  confidence: number;              // 0-100
  gateResults: GateCheck[];
  explanation: string;
  isFlip: boolean;
  riskFlags: string[];
}
```

S-Curve

```
export interface SCurveInput {
  items: { amount: number; startMonth: number; endMonth: number;
           curveType: "s-curve" | "linear" | "front-loaded" | "back-loaded"; }[];
  totalMonths: number;
}
export interface SCurveOutput {
  monthlyCashflows: number[];      // length = totalMonths
  cumulativeCashflows: number[];
  totalAmount: number;
}
```

2.4 View DTOs (dto/views.ts)

Pre-aggregated shapes returned by the API for each MVP screen. The UI fetches these directly — no client-side joining.

DealDashboardView

```
export interface DealDashboardView {
  deal: Pick<Deal, "id" | "name" | "status" | "lifecyclePhase" | "currentMonth">;
  property: Pick<Property, "location" | "keys" | "starRating">;
```

```
  partnership: Partnership;
  latestRecommendation: RecommendationState | null;
  latestProforma: { scenarioKey: string; irr: number; npv: number;
    equityMultiple: number; paybackYear: number; } | null;
  latestMC: { p50Irr: number; probNpvNeg: number; } | null;
  latestFactor: { compositeScore: number; impliedDiscountRate: number; } | null;
  budgetSummary: { totalBudget: number; spent: number; forecast: number;
    variancePct: number; } | null;
  constructionProgress: { pctComplete: number; daysAheadBehind: number;
    openRFIs: number; pendingCOs: number; } | null;
  recentEvents: { event: string; timestamp: string; actor: string; }[];
  recommendationHistory: { version: number; verdict: string;
    confidence: number; timestamp: string; }[];
}
```

ScenarioExplorerView

```
export interface ScenarioExplorerView {
  dealId: string;
  scenarios: {
    key: string; name: string; probability: number;
    proforma: ProFormaOutput;
  }[];
  mcSummary: MCOutput;
  factorSummary: FactorScoreOutput;
  comparisonTable: {               // pivoted for easy table rendering
    metric: string;
    bear: number; base: number; bull: number;
  }[];
}
```

BudgetTrackerView

```
export interface BudgetTrackerView {
  dealId: string;
  asOfMonth: number;
  summary: BudgetAnalysisOutput;
  sCurveChart: { month: number; planned: number; actual: number;
    forecast: number; }[];
  topVariances: { costCode: string; description: string;
    budget: number; forecast: number; variance: number; pct: number; }[];
  rfis: RFI[];
  changeOrders: ChangeOrder[];
  riskExposure: { category: string; count: number;
    totalExposureCr: number; }[];
}
```

2.5 Domain Events (events/event-types.ts)

```
export type DomainEvent =
  | { type: "assumption.updated"; dealId: string; userId: string;
      field: string; oldValue: unknown; newValue: unknown; }
  | { type: "macro.refreshed"; dealId: string; indicators: Record<string, number>; }
  | { type: "phase.advanced"; dealId: string; from: string; to: string; userId: string; }
  | { type: "milestone.completed"; dealId: string; milestoneId: string; userId: string; }
  | { type: "milestone.delayed"; dealId: string; milestoneId: string; delayDays: number; }
  | { type: "budget.actual.updated"; dealId: string; month: number; userId: string; }
  | { type: "rfi.created"; dealId: string; rfiId: string; userId: string; }
  | { type: "rfi.resolved"; dealId: string; rfiId: string; userId: string; }
  | { type: "change-order.submitted"; dealId: string; coId: string; userId: string; }
  | { type: "change-order.approved"; dealId: string; coId: string; approverUserId: string; }
  | { type: "risk.escalated"; dealId: string; riskId: string; userId: string; }
```

```
  | { type: "recommendation.changed"; dealId: string; from: string; to: string; version:
number; }
  | { type: "engine.completed"; dealId: string; engineName: string; version: number;
      durationMs: number; };

// Each event includes a standard envelope:
export interface EventEnvelope<T extends DomainEvent = DomainEvent> {
  id: string;          // UUID v7
  timestamp: string;   // ISO 8601
  source: string;      // "api" | "workflow" | "system"
  payload: T;
}
```

3. Engine Skeletons (packages/engines)

Every engine is a pure function: (input) => output. No I/O, no database, no HTTP. This section provides the module interface and algorithmic stubs for each of the 6 MVP engines.

3.1 File Layout

```
packages/engines/src/
  index.ts                # barrel: export { factor, underwriter, mc, ... }
  factor/index.ts
  underwriter/index.ts
  montecarlo/index.ts
  budget/index.ts
  decision/index.ts
  scurve/index.ts
  _shared/                # internal helpers: IRR, NPV, percentile, triangular
    irr.ts
    npv.ts
    distributions.ts
    percentile.ts
```

3.2 Shared Math Utilities (_shared/)

irr.ts — Newton-Raphson IRR

```
// Newton-Raphson IRR solver.
// cashFlows[0] is typically negative (initial equity outlay).
// Returns annualized IRR as a decimal (e.g. 0.157 = 15.7%).
//
// Algorithm:
//   1. Start with guess = 0.10
//   2. For each iteration (max 100):
//      a. Compute NPV at current guess
//      b. Compute derivative of NPV w.r.t. rate
//      c. Update guess: guess -= npv / dNpv
//      d. If |npv| < 1e-7, converge
//   3. Return guess or NaN if no convergence
export function calcIRR(cashFlows: number[], guess = 0.10): number {
  // TODO: implement Newton-Raphson loop
  throw new Error("not implemented");
}
```

npv.ts — Discounted NPV

```
// Standard NPV: sum of cashFlows[t] / (1 + rate)^t
export function calcNPV(cashFlows: number[], rate: number): number {
  return cashFlows.reduce((sum, cf, t) => sum + cf / Math.pow(1 + rate, t), 0);
}
```

distributions.ts — Random Samplers

```
// Box-Muller normal random (mean=0, std=1)
export function normalRandom(rng: () => number = Math.random): number { ... }

// Triangular distribution sampler
export function triangular(min: number, mode: number, max: number,
                           rng: () => number = Math.random): number { ... }

// Seeded PRNG (xoshiro128** or similar) for reproducible MC runs
export function createSeededRng(seed: number): () => number { ... }
```

3.3 Factor Engine (factor/index.ts)

```
import { FactorScoreInput, FactorScoreOutput, FactorDetail } from "@v3grand/core";

export function scoreFactors(input: FactorScoreInput): FactorScoreOutput {
  // 1. Score each factor within each domain (1.0 - 5.0)
  //    Global (20%):  repo rate, inflation, GDP growth, tourism index, regulatory
  //    Local  (25%):  city RevPAR, airport traffic, infra pipeline, competitor supply,
demand drivers
  //    Asset  (35%):  location score, brand/flag, room mix, amenity set, construction risk,
design quality
  //    Sponsor(20%):  track record, liquidity, local relationships, team depth
  //
  // 2. Weight factors within domain, produce domainScore
  // 3. Composite = sum(domainScore * domainWeight)
  // 4. ImpliedDiscountRate = riskFreeRate + 0.05 + (5.0 - composite) * 0.02
  // 5. ImpliedCapRate = impliedDiscountRate - growthRate
  //
  // Each factor maps to a scoring function:
  //   scoreFn(deal, macroIndicators?) => { score: number, rationale: string }
  // For MVP, these are deterministic rule-based scorers.
  // Post-MVP, swap in ML-based scorers.

  const global = scoreGlobalDomain(input);   // returns { score, factors[] }
  const local  = scoreLocalDomain(input);
  const asset  = scoreAssetDomain(input);
  const sponsor= scoreSponsorDomain(input);

  const composite = global.score * 0.20 + local.score * 0.25
                  + asset.score * 0.35 + sponsor.score * 0.20;

  const rf = input.deal.financialAssumptions.riskFreeRate;
  const impliedDiscountRate = rf + 0.05 + (5.0 - composite) * 0.02;
  const impliedCapRate = impliedDiscountRate - input.deal.financialAssumptions.inflationRate;

  return {
    compositeScore: Math.round(composite * 100) / 100,
    domainScores: {
      global:  { score: global.score, weight: 0.20, factors: global.factors },
      local:   { score: local.score,  weight: 0.25, factors: local.factors },
      asset:   { score: asset.score,  weight: 0.35, factors: asset.factors },
      sponsor: { score: sponsor.score,weight: 0.20, factors: sponsor.factors },
    },
    impliedDiscountRate,
    impliedCapRate,
  };
}

// --- Domain scorers (stubs) ---
function scoreGlobalDomain(input: FactorScoreInput)
  : { score: number; factors: FactorDetail[] } {
  // TODO: implement 5-6 global factors
  // Example: repoRateFactor: score = clamp(1, 5, 5 - (repoRate - 0.05) * 20)
  throw new Error("not implemented");
}
// scoreLocalDomain, scoreAssetDomain, scoreSponsorDomain: same pattern
```

3.4 Underwriter / ProForma (underwriter/index.ts)

This is the largest engine. The pseudocode below outlines the full 10-year cash-flow model.

```
import { ProFormaInput, ProFormaOutput, YearProjection } from "@v3grand/core";
import { calcIRR } from "../_shared/irr";
import { calcNPV } from "../_shared/npv";

export function buildProForma(input: ProFormaInput): ProFormaOutput {
  const { deal, scenarioKey, overrides } = input;
  const fin = { ...deal.financialAssumptions, ...overrides };
  const scenario = deal.scenarios[scenarioKey];
  const mkt = deal.marketAssumptions;

  // --- Step 1: Compute total investment and equity ---
  // totalInv = phase1.totalBudgetCr + (scenario.phase2Trigger ? phase2.totalBudgetCr : 0)
  // equityInv = totalInv * fin.equityRatio
  // debtDrawdown = totalInv - equityInv

  // --- Step 2: Build year-by-year projections (years 1-10) ---
  // For each year Y (1..10):
  //   keys = phase1Keys + (phase2Triggered && Y >= phase2StartYear ? phase2Keys : 0)
  //   occupancy = lerp(occupancyRamp, stabilize at scenario.occupancyStabilized)
  //   adr = scenario.adrStabilized * (1 + fin.inflationRate)^(Y-stabYear)
  //   revpar = occupancy * adr
  //   roomRevenue = revpar * keys * 365
  //   totalRevenue = roomRevenue / mkt.revenueMix.rooms
  //   departmentalProfit = sum(dept.revenue * (1 - dept.costPct)) across USALI depts
  //   undistributed = sum(undistributed expenses)
  //   GOP = departmentalProfit - undistributed
  //   EBITDA = GOP - managementFee - incentiveFee - ffneReserve
  //   debtService = annuity(debtDrawdown, interestRate, tenorYears)
  //   FCFE = EBITDA * (1 - taxRate) - debtService

  // --- Step 3: Build cash flow array ---
  // cashFlows[0] = -equityInv (negative)
  // cashFlows[1..9] = FCFE for years 1-9
  // cashFlows[10] = FCFE[10] + exitEquityValue
  // exitValue = EBITDA[10] * fin.exitMultiple
  // exitEquityValue = exitValue - outstandingDebt

  // --- Step 4: Compute metrics ---
  // irr = calcIRR(cashFlows)
  // npv = calcNPV(cashFlows, fin.wacc)
  // equityMultiple = (sum of positive cashFlows) / equityInv
  // paybackYear = first Y where cumulative FCFE >= equityInv
  // avgDSCR = mean(EBITDA[Y] / debtService[Y]) for all years with debt

  const years: YearProjection[] = [];
  // TODO: implement the year-by-year loop per pseudocode above

  const cashFlows: number[] = [];
  // TODO: build from years[] + exit value

  return {
    scenarioKey,
    years,
    totalInvestment: 0,  // TODO
    equityInvestment: 0,
    debtDrawdown: 0,
    cashFlows,
```

```
    irr: calcIRR(cashFlows),
    npv: calcNPV(cashFlows, fin.wacc),
    equityMultiple: 0,
    paybackYear: 0,
    avgDSCR: 0,
    exitValue: 0,
  };
}
```

3.5 Monte Carlo (montecarlo/index.ts)

```
import { MCInput, MCOutput } from "@v3grand/core";
import { buildProForma } from "../underwriter";
import { triangular, normalRandom, createSeededRng } from "../_shared/distributions";
import { percentiles } from "../_shared/percentile";

export function runMonteCarlo(input: MCInput): MCOutput {
  const { deal, iterations = 5000, seed } = input;
  const rng = seed != null ? createSeededRng(seed) : Math.random;

  // --- For each iteration i (0..iterations-1): ---
  //   1. Sample perturbed scenario:
  //      occStab   = triangular(0.60, base.occupancyStab, 0.88, rng)
  //      adrStab   = exp(ln(base.adrStab) + 0.10 * normalRandom(rng))
  //      exitMult  = triangular(9, base.exitMultiple, 16, rng)
  //      ebitdaAdj = triangular(0.34, base.ebitdaMargin, 0.45, rng)
  //      phase2    = occStab > 0.70 && rng() < 0.75
  //
  //   2. Inject perturbed scenario into deal.scenarios._mc
  //   3. Call buildProForma(deal, "_mc") to get full cash-flow model
  //   4. Collect IRR, NPV from result
  //   5. Delete deal.scenarios._mc (restore deal)
  //
  // --- After all iterations: ---
  //   Compute percentiles (p5, p10, p25, p50, p75, p90, p95) for IRR and NPV
  //   probNpvNegative = count(npv < 0) / iterations
  //   probIrrBelowWacc = count(irr < deal.financialAssumptions.wacc) / iterations
  //   Build histogram buckets for UI
  //   Compute sensitivity ranking via partial correlation

  const results: { irr: number; npv: number }[] = [];
  // TODO: implement sampling loop reusing buildProForma()

  const irrs = results.map(r => r.irr);
  const npvs = results.map(r => r.npv);

  return {
    iterations,
    irrDistribution: percentiles(irrs),
    npvDistribution: percentiles(npvs),
    probNpvNegative: npvs.filter(n => n < 0).length / iterations,
    probIrrBelowWacc: irrs.filter(i => i < deal.financialAssumptions.wacc).length /
iterations,
    probPhase2Trigger: 0,           // TODO: track from sampling loop
    sensitivityRanking: [],          // TODO: partial correlation analysis
    histogram: [],                   // TODO: bucket into 50 bins
  };
}
```

3.6 Budget Variance (budget/index.ts)

```
import { BudgetAnalysisInput, BudgetAnalysisOutput } from "@v3grand/core";

export function analyzeBudget(input: BudgetAnalysisInput): BudgetAnalysisOutput {
  const { deal, asOfMonth } = input;
  // 1. Aggregate CapexLineItems up to asOfMonth:
  //    totalBudget, totalCommitted, totalSpent, totalForecast
  // 2. Group by category and costCode, compute variance for each
  // 3. Build S-curve data: for each month 0..totalMonths,
  //    { planned (from original S-curve), actual (from budget lines), forecast }
  // 4. Generate alerts:
  //    - Any costCode where |variance| > 10% of budget => AMBER
  //    - Any costCode where |variance| > 20% of budget => RED
  //    - Total forecast > totalBudget * 1.05 => BUDGET OVERRUN alert
  //    - Committed > totalBudget * 0.95 && spent < committed * 0.50 => CASH FLOW RISK

  // TODO: implement aggregation and alert logic
  throw new Error("not implemented");
}
```

3.7 Decision Engine (decision/index.ts)

```
import { DecisionInput, DecisionOutput, GateCheck,
         RecommendationVerdict } from "@v3grand/core";

export function evaluate(input: DecisionInput): DecisionOutput {
  const { deal, factorResult, proformaResults, mcResult,
          budgetResult, currentRecommendation } = input;
  const fin = deal.financialAssumptions;
  const base = proformaResults["base"];

  // --- Gate checks ---
  const gates: GateCheck[] = [
    gate("IRR > WACC + 200bps", base.irr, fin.wacc + 0.02, base.irr > fin.wacc + 0.02),
    gate("NPV > 0", base.npv, 0, base.npv > 0),
    gate("Equity Multiple > 1.8x", base.equityMultiple, 1.8, base.equityMultiple > 1.8),
    gate("DSCR > 1.3x", base.avgDSCR, 1.3, base.avgDSCR > 1.3),
    gate("P(NPV < 0) < 20%", mcResult.probNpvNegative, 0.20,
         mcResult.probNpvNegative < 0.20),
    gate("Factor Score > 3.0", factorResult.compositeScore, 3.0,
         factorResult.compositeScore > 3.0),
    gate("MC P10 IRR > 5%", mcResult.irrDistribution.p10, 0.05,
         mcResult.irrDistribution.p10 > 0.05),
  ];

  // If in construction phase and budget data available, add budget gates
  if (budgetResult) {
    gates.push(gate("Budget Variance < 10%",
      Math.abs(budgetResult.varianceToOriginal / budgetResult.totalBudget),
      0.10, Math.abs(budgetResult.varianceToOriginal / budgetResult.totalBudget) < 0.10));
  }

  // --- Verdict logic ---
  // passCount / totalGates => passRate
  // passRate >= 0.85 => INVEST
  // passRate >= 0.70 => HOLD
  // passRate >= 0.50 => DE-RISK
  // passRate >= 0.30 => EXIT
```

```
  // passRate < 0.30  => DO-NOT-PROCEED
  // Confidence = passRate * 100, adjusted by MC spread (p90-p10)

  const passRate = gates.filter(g => g.passed).length / gates.length;
  const verdict = deriveVerdict(passRate);
  const confidence = computeConfidence(passRate, mcResult);
  const prev = currentRecommendation?.verdict ?? null;

  return {
    verdict,
    confidence: Math.round(confidence),
    gateResults: gates,
    explanation: buildExplanation(verdict, gates, passRate),
    isFlip: prev != null && verdict !== prev,
    riskFlags: extractRiskFlags(gates, mcResult, budgetResult),
  };
}

function gate(name: string, actual: number, threshold: number, passed: boolean): GateCheck {
  return { name, actual, threshold, passed };
}
function deriveVerdict(passRate: number): RecommendationVerdict { /* TODO */ }
function computeConfidence(passRate: number, mc: MCOutput): number { /* TODO */ }
function buildExplanation(...args: any[]): string { /* TODO */ }
function extractRiskFlags(...args: any[]): string[] { /* TODO */ }
```

3.8 S-Curve (scurve/index.ts)

```
import { SCurveInput, SCurveOutput } from "@v3grand/core";

export function distribute(input: SCurveInput): SCurveOutput {
  const { items, totalMonths } = input;
  const monthly = new Array(totalMonths).fill(0);

  for (const item of items) {
    const span = item.endMonth - item.startMonth;
    for (let m = item.startMonth; m < item.endMonth && m < totalMonths; m++) {
      const t = (m - item.startMonth) / span; // 0..1 progress
      let weight: number;
      switch (item.curveType) {
        case "s-curve":
          // Logistic S-curve: w(t) = 1 / (1 + exp(-12*(t - 0.5)))
          // Differentiate for monthly weight, normalize
          weight = sCurveWeight(t, span);
          break;
        case "linear":
          weight = 1 / span;
          break;
        case "front-loaded":
          // Beta(2, 5) shaped: heavier early
          weight = betaPdfWeight(t, 2, 5, span);
          break;
        case "back-loaded":
          // Beta(5, 2) shaped: heavier late
          weight = betaPdfWeight(t, 5, 2, span);
          break;
      }
      monthly[m] += item.amount * weight;
    }
  }
```

```
  const cumulative = monthly.reduce((acc, v) => {
    acc.push((acc.length ? acc[acc.length - 1] : 0) + v);
    return acc;
  }, [] as number[]);

  return {
    monthlyCashflows: monthly,
    cumulativeCashflows: cumulative,
    totalAmount: monthly.reduce((a, b) => a + b, 0),
  };
}

function sCurveWeight(t: number, span: number): number { /* TODO */ }
function betaPdfWeight(t: number, a: number, b: number, span: number): number { /* TODO */ }
```

3.9 Testing Strategy

Each engine gets a dedicated test file (e.g. factor/index.test.ts). Tests use fixture deals loaded from a shared __fixtures__/v3grand-deal.json file.

Engine	Key Test Cases	Golden Values Source
Factor	Known deal => composite 3.7, implied discount 14.6%. Edge: all factors=1 => composite=1.	Manually scored from Excel assumptions
Underwriter	Base scenario => IRR ~15.7%, NPV > 0. Bear => IRR ~10%. Bull => IRR ~22%. Verify DSCR > 1.3 in base.	Excel EBITDA & CF sheet
Monte Carlo	Seeded run (seed=42, 1000 iter) => deterministic percentiles. P50 within 1% of base IRR. P(NPV<0) < 15%.	Regression against v3engine.html results
Budget	No variance when actuals = planned. 15% overrun flagged as RED. S-curve cumulative matches total.	CAPEX & Phasing sheet
Decision	85% gates pass => INVEST. 50% pass => DE-RISK. Flip detection when previous was INVEST, new is HOLD.	Gate thresholds from blueprint
S-Curve	Linear over 12 months => each month = total/12. S-curve cumulative is monotonically increasing.	Manual verification

4. API & Workflow Wiring

4.1 Fastify Server Structure (packages/api)

```
packages/api/src/
  index.ts               # Fastify app bootstrap, plugin registration
  plugins/
    auth.ts              # Clerk/JWT verification, role extraction
    rbac.ts              # Permission middleware (role x module x action)
    websocket.ts         # @fastify/websocket for real-time events
    error-handler.ts     # Structured error responses
  routes/
    deals.ts             # CRUD + lifecycle advancement
    engines/
      factor.ts          # POST /deals/:id/engines/factor
      underwriter.ts     # POST /deals/:id/engines/underwriter
      montecarlo.ts      # POST /deals/:id/engines/montecarlo
      budget.ts          # POST /deals/:id/engines/budget
      decision.ts        # POST /deals/:id/engines/decision, GET .../history
      scurve.ts          # POST /deals/:id/engines/scurve
    views/
      dashboard.ts       # GET /deals/:id/dashboard
      scenarios.ts       # GET /deals/:id/scenarios
      budget-tracker.ts  # GET /deals/:id/budget
    construction/
      rfis.ts            # CRUD for RFIs
      change-orders.ts   # CRUD + submit/approve
    audit.ts             # GET /deals/:id/audit
  services/
    event-publisher.ts   # Emit DomainEvent to NATS JetStream
    engine-runner.ts     # Load deal, call engine, persist result
```

4.2 Route Specifications

Every engine route follows the same pattern. Below is the canonical shape, then a table of all routes.

Canonical Engine Route Pattern

```
// POST /deals/:dealId/engines/factor
async function factorRoute(app: FastifyInstance) {
  app.post<{ Params: { dealId: string } }>(
    "/deals/:dealId/engines/factor",
    { preHandler: [requireAuth, requireRole("lead-investor", "co-investor")] },
    async (req, reply) => {
      const deal = await dealRepo.findById(req.params.dealId);
      if (!deal) return reply.code(404).send({ error: "Deal not found" });

      const input: FactorScoreInput = {
        deal,
        macroIndicators: req.body.macroIndicators, // optional override
      };

      const output = scoreFactors(input);  // pure engine call

      await engineResultRepo.insert({
        dealId: deal.id,
        engineName: "factor",
        version: await engineResultRepo.nextVersion(deal.id, "factor"),
        input, output,
```

```
          durationMs: elapsed,
          triggeredBy: "api.manual",
        });

        await eventPublisher.publish({
          type: "engine.completed",
          dealId: deal.id,
          engineName: "factor",
          version: result.version,
          durationMs: elapsed,
        });

        return reply.send(output);
      }
    );
  }
```

Full Route Table

Method	Path	Auth/Role	Returns
GET	/deals	Any authenticated	Deal[] (summary list)
GET	/deals/:id	Any deal member	Deal (full)
PATCH	/deals/:id	lead-investor, operator	Deal (updated)
POST	/deals/:id/advance-phase	lead-investor	{ phase, recommendation }
POST	/deals/:id/engines/factor	lead-investor, co-investor	FactorScoreOutput
POST	/deals/:id/engines/underwriter	lead-investor, co-investor	ProFormaOutput (per scenario)
POST	/deals/:id/engines/montecarlo	lead-investor, co-investor	MCOutput
POST	/deals/:id/engines/budget	lead-investor, operator, PM	BudgetAnalysisOutput
POST	/deals/:id/engines/decision	lead-investor	DecisionOutput
POST	/deals/:id/engines/scurve	lead-investor, operator	SCurveOutput
GET	/deals/:id/engines/:engine/history	Any deal member	EngineResult[]
GET	/deals/:id/dashboard	Any deal member	DealDashboardView
GET	/deals/:id/scenarios	lead-investor, co-investor	ScenarioExplorerView
GET	/deals/:id/budget	lead-investor, operator, PM	BudgetTrackerView
GET	/deals/:id/decision/history	Any deal member	RecommendationState[]
GET	/deals/:id/decision/explain/:version	Any deal member	RecommendationState (single)
POST	/deals/:id/rfis	operator, PM	RFI
PATCH	/deals/:id/rfis/:rfiId	operator, PM	RFI (updated)
POST	/deals/:id/change-orders	operator, PM	ChangeOrder
POST	/deals/:id/change-orders/:coId/submit	operator, PM	ChangeOrder (submitted)
POST	/deals/:id/change-orders/:coId/approve	lead-investor	ChangeOrder (approved/rejected)
GET	/deals/:id/audit	lead-investor	AuditLogEntry[]

4.3 RBAC Middleware

```
// plugins/rbac.ts
```

```
type Role = "lead-investor" | "co-investor" | "operator" | "lender" | "pm";

function requireRole(...allowed: Role[]) {
  return async (req: FastifyRequest, reply: FastifyReply) => {
    const userRole = req.user.role; // extracted by auth plugin from JWT
    if (!allowed.includes(userRole)) {
      return reply.code(403).send({ error: "Insufficient permissions" });
    }
  };
}

// Permission matrix (from blueprint) enforced here.
// For MVP, role is a single string claim in the JWT.
// Post-MVP: per-deal role assignments with deal membership.
```

4.4 Temporal Workflows (packages/workflows)

```
packages/workflows/src/
  index.ts
  workflows/
    monthly-revaluation.ts
    change-order-approval.ts
    on-assumption-change.ts
  activities/
    engine-activities.ts     # wrappers that call engine functions + persist
    notification-activities.ts
    persistence-activities.ts
  worker.ts                  # Temporal worker bootstrap
```

4.4.1 MonthlyRevaluation Workflow

```
import { proxyActivities } from "@temporalio/workflow";
import type { EngineActivities, NotificationActivities } from "../activities";

const engine = proxyActivities<EngineActivities>({
  startToCloseTimeout: "60s",
  retry: { maximumAttempts: 3, backoffCoefficient: 2 },
});
const notify = proxyActivities<NotificationActivities>({
  startToCloseTimeout: "10s",
});

export async function monthlyRevaluation(dealId: string): Promise<void> {
  // Step 1: Refresh macro indicators (or use latest cached)
  const macro = await engine.refreshMacroIndicators(dealId);

  // Step 2: Run Factor Engine with fresh macro data
  const factorResult = await engine.runFactorEngine(dealId, macro);

  // Step 3: Run Underwriter for all 3 scenarios in parallel
  const [bear, base, bull] = await Promise.all([
    engine.runUnderwriter(dealId, "bear"),
    engine.runUnderwriter(dealId, "base"),
    engine.runUnderwriter(dealId, "bull"),
  ]);

  // Step 4: Run Monte Carlo (depends on Underwriter being current)
  const mcResult = await engine.runMonteCarlo(dealId);
```

```
  // Step 5: Run Decision Engine (aggregates all upstream results)
  const decision = await engine.runDecisionEngine(dealId);

  // Step 6: If recommendation flipped, alert stakeholders
  if (decision.isFlip) {
    await notify.alertRecommendationFlip(dealId, decision);
  }

  // Step 7: If any risk flags, notify relevant roles
  if (decision.riskFlags.length > 0) {
    await notify.alertRiskFlags(dealId, decision.riskFlags);
  }
}
```

4.4.2 ChangeOrderApproval Workflow

```
import { proxyActivities, condition, sleep } from "@temporalio/workflow";

interface COSignals { approve: boolean; approverUserId: string; }

export async function changeOrderApproval(
  dealId: string, changeOrderId: string
): Promise<void> {
  let approvalDecision: COSignals | null = null;

  // Step 1: Notify approver(s) that a CO is pending
  await notify.sendCOApprovalRequest(dealId, changeOrderId);

  // Step 2: Wait for approval signal (timeout: 7 days)
  const approved = await condition(
    () => approvalDecision !== null,
    "7 days"
  );

  if (!approved || !approvalDecision) {
    // Timeout: escalate to lead-investor
    await notify.escalateCOTimeout(dealId, changeOrderId);
    return;
  }

  if (approvalDecision.approve) {
    // Step 3a: Approved → rebase budget
    await persist.approveChangeOrder(dealId, changeOrderId, approvalDecision.approverUserId);
    await engine.runBudgetEngine(dealId);
    await engine.runSCurve(dealId);   // rebase S-curve with new amounts
    await engine.runDecisionEngine(dealId); // cascade to recommendation
  } else {
    // Step 3b: Rejected → mark rejected, no engine cascade
    await persist.rejectChangeOrder(dealId, changeOrderId, approvalDecision.approverUserId);
  }

  await notify.sendCODecisionNotification(dealId, changeOrderId, approvalDecision.approve);
}
```

4.4.3 Activity Interfaces

```
// activities/engine-activities.ts
export interface EngineActivities {
  refreshMacroIndicators(dealId: string): Promise<MacroIndicators>;
  runFactorEngine(dealId: string, macro?: MacroIndicators): Promise<FactorScoreOutput>;
```

```
  runUnderwriter(dealId: string, scenario: string): Promise<ProFormaOutput>;
  runMonteCarlo(dealId: string): Promise<MCOutput>;
  runBudgetEngine(dealId: string): Promise<BudgetAnalysisOutput>;
  runDecisionEngine(dealId: string): Promise<DecisionOutput>;
  runSCurve(dealId: string): Promise<SCurveOutput>;
}

// Each activity implementation follows this pattern:
// 1. Load current deal snapshot from DB
// 2. Load latest upstream engine results from DB
// 3. Call pure engine function with assembled input
// 4. Persist EngineResult row (versioned)
// 5. Publish engine.completed event to NATS
// 6. Return output
```

4.4.4 Temporal Worker Bootstrap

```
// worker.ts
import { Worker } from "@temporalio/worker";
import * as activities from "./activities";

async function run() {
  const worker = await Worker.create({
    workflowsPath: require.resolve("./workflows"),
    activities,
    taskQueue: "v3grand-main",
    maxConcurrentWorkflowTaskExecutions: 10,
    maxConcurrentActivityTaskExecutions: 20,
  });
  await worker.run();
}
run().catch(err => { console.error(err); process.exit(1); });
```

4.5 Event Flow Diagram (Text)

The following shows how a user action cascades through the system:

```
User updates assumption in UI
  │
  ├─→ PATCH /deals/:id          (API persists change, emits event)
  │     │
  │     └─→ NATS: assumption.updated
  │           │
  │           └─→ Temporal: starts OnAssumptionChange workflow
  │                 │
  │                 ├─→ Activity: runFactorEngine(dealId)
  │                 ├─→ Activity: runUnderwriter(dealId, "bear"|"base"|"bull")
  │                 ├─→ Activity: runMonteCarlo(dealId)
  │                 ├─→ Activity: runDecisionEngine(dealId)
  │                 │     │
  │                 │     └─→ if isFlip: notify stakeholders
  │                 │
  │                 └─→ NATS: engine.completed (x4)
  │                       │
  │                       └─→ WebSocket: push to connected UI clients
  │
  └─→ UI auto-refetches via React Query invalidation
```

5. Database Layer (packages/db)

5.1 File Layout

```
packages/db/src/
  index.ts               # barrel: export schema, client, queries
  client.ts              # Drizzle + pg pool initialization
  schema/
    deals.ts
    deal-snapshots.ts
    budget-lines.ts
    rfis.ts
    change-orders.ts
    engine-results.ts
    recommendations.ts
    audit-log.ts
    index.ts             # re-export all tables
  queries/
    deal-queries.ts
    engine-queries.ts
    view-queries.ts      # dashboard, scenarios, budget-tracker
  migrations/
    0001_initial.sql
  seed/
    v3grand.ts           # seed script with V3 Grand deal data
```

5.2 Drizzle Table Definitions

All tables use Drizzle ORM with PostgreSQL dialect. UUIDs are v7 (time-sortable). JSONB columns store engine-specific payloads.

deals

```
import { pgTable, uuid, varchar, integer, jsonb, timestamp } from "drizzle-orm/pg-core";

export const deals = pgTable("deals", {
  id:             uuid("id").primaryKey().defaultRandom(),
  name:           varchar("name", { length: 255 }).notNull(),
  assetClass:     varchar("asset_class", { length: 50 }).notNull().default("hotel"),
  status:         varchar("status", { length: 20 }).notNull().default("draft"),
  lifecyclePhase: varchar("lifecycle_phase", { length: 50 }).notNull().default("pre-
development"),
  currentMonth:   integer("current_month").notNull().default(0),
  version:        integer("version").notNull().default(1),
  property:       jsonb("property").notNull(),
  partnership:    jsonb("partnership").notNull(),
  marketAssumptions:    jsonb("market_assumptions").notNull(),
  financialAssumptions: jsonb("financial_assumptions").notNull(),
  capexPlan:      jsonb("capex_plan").notNull(),
  opexModel:      jsonb("opex_model").notNull(),
  scenarios:      jsonb("scenarios").notNull(),
  createdAt:      timestamp("created_at").notNull().defaultNow(),
  updatedAt:      timestamp("updated_at").notNull().defaultNow(),
});
```

Design decision: JSONB for nested objects (property, assumptions, etc.) rather than normalized tables. At MVP scale (single deal), this avoids join complexity. The Zod validators in @v3grand/core enforce structure on read/write.

deal_snapshots (immutable history)

```
export const dealSnapshots = pgTable("deal_snapshots", {
  id:        uuid("id").primaryKey().defaultRandom(),
  dealId:    uuid("deal_id").notNull().references(() => deals.id),
  version:   integer("version").notNull(),
  snapshot:  jsonb("snapshot").notNull(),  // full Deal object at this version
  changedBy: varchar("changed_by", { length: 255 }).notNull(),
  changeReason: varchar("change_reason", { length: 500 }),
  createdAt: timestamp("created_at").notNull().defaultNow(),
});
// Index: (deal_id, version) UNIQUE
```

engine_results (versioned, append-only)

```
export const engineResults = pgTable("engine_results", {
  id:          uuid("id").primaryKey().defaultRandom(),
  dealId:      uuid("deal_id").notNull().references(() => deals.id),
  engineName:  varchar("engine_name", { length: 50 }).notNull(),
  version:     integer("version").notNull(),
  input:       jsonb("input").notNull(),
  output:      jsonb("output").notNull(),
  durationMs:  integer("duration_ms").notNull(),
  triggeredBy: varchar("triggered_by", { length: 100 }).notNull(),
  createdAt:   timestamp("created_at").notNull().defaultNow(),
});
// Index: (deal_id, engine_name, version) UNIQUE
// Index: (deal_id, engine_name, created_at DESC) for "get latest"
```

recommendations (versioned, append-only)

```
export const recommendations = pgTable("recommendations", {
  id:               uuid("id").primaryKey().defaultRandom(),
  dealId:           uuid("deal_id").notNull().references(() => deals.id),
  version:          integer("version").notNull(),
  verdict:          varchar("verdict", { length: 20 }).notNull(),
  confidence:       integer("confidence").notNull(),
  triggerEvent:     varchar("trigger_event", { length: 100 }).notNull(),
  factorSnapshot:   jsonb("factor_snapshot").notNull(),
  proformaSnapshot: jsonb("proforma_snapshot").notNull(),
  mcSnapshot:       jsonb("mc_snapshot").notNull(),
  gateResults:      jsonb("gate_results").notNull(),
  explanation:      varchar("explanation", { length: 2000 }).notNull(),
  previousVerdict:  varchar("previous_verdict", { length: 20 }),
  isFlip:           varchar("is_flip", { length: 5 }).notNull().default("false"),
  createdAt:        timestamp("created_at").notNull().defaultNow(),
});
// Index: (deal_id, version) UNIQUE
// Index: (deal_id, created_at DESC) for "get latest"
```

budget_lines, rfis, change_orders

```
export const budgetLines = pgTable("budget_lines", {
  id:          uuid("id").primaryKey().defaultRandom(),
  dealId:      uuid("deal_id").notNull().references(() => deals.id),
  costCode:    varchar("cost_code", { length: 50 }).notNull(),
  month:       integer("month").notNull(),
  planned:     integer("planned").notNull(),       // INR (integer for precision)
  committed:   integer("committed").notNull().default(0),
  actual:      integer("actual").notNull().default(0),
  forecast:    integer("forecast").notNull().default(0),
  updatedAt:   timestamp("updated_at").notNull().defaultNow(),
});
// Index: (deal_id, cost_code, month) UNIQUE
```

```
export const rfis = pgTable("rfis", {
  id:          uuid("id").primaryKey().defaultRandom(),
  dealId:      uuid("deal_id").notNull().references(() => deals.id),
  number:      integer("number").notNull(),
  title:       varchar("title", { length: 500 }).notNull(),
  description: varchar("description", { length: 5000 }),
  priority:    varchar("priority", { length: 20 }).notNull().default("medium"),
  status:      varchar("status", { length: 20 }).notNull().default("open"),
  assigneeId:  varchar("assignee_id", { length: 255 }),
  costImpact:  integer("cost_impact"),
  scheduleImpactDays: integer("schedule_impact_days"),
  createdAt:   timestamp("created_at").notNull().defaultNow(),
  resolvedAt:  timestamp("resolved_at"),
});

export const changeOrders = pgTable("change_orders", {
  id:          uuid("id").primaryKey().defaultRandom(),
  dealId:      uuid("deal_id").notNull().references(() => deals.id),
  number:      integer("number").notNull(),
  title:       varchar("title", { length: 500 }).notNull(),
  description: varchar("description", { length: 5000 }),
  costImpact:  integer("cost_impact").notNull(),
  scheduleImpactDays: integer("schedule_impact_days").notNull().default(0),
  status:      varchar("status", { length: 20 }).notNull().default("draft"),
  approverRole: varchar("approver_role", { length: 50 }),
  submittedBy: varchar("submitted_by", { length: 255 }),
  approvedBy:  varchar("approved_by", { length: 255 }),
  affectedCostCodes: jsonb("affected_cost_codes").default([]),
  createdAt:   timestamp("created_at").notNull().defaultNow(),
  decidedAt:   timestamp("decided_at"),
});
```

audit_log

```
export const auditLog = pgTable("audit_log", {
  id:         uuid("id").primaryKey().defaultRandom(),
  timestamp:  timestamp("timestamp").notNull().defaultNow(),
  dealId:     uuid("deal_id").notNull().references(() => deals.id),
  userId:     varchar("user_id", { length: 255 }).notNull(),
  role:       varchar("role", { length: 50 }).notNull(),
  module:     varchar("module", { length: 50 }).notNull(),
  action:     varchar("action", { length: 100 }).notNull(),
  entityType: varchar("entity_type", { length: 50 }),
  entityId:   varchar("entity_id", { length: 255 }),
  diff:       jsonb("diff"),
});
// Index: (deal_id, timestamp DESC)
// Index: (deal_id, module, action)
```

5.3 Versioning Patterns

Insert new version: For engine_results and recommendations, every run creates a new row. The version column is monotonically increasing per (dealId, engineName).

```
// queries/engine-queries.ts
export async function insertEngineResult(db: DrizzleClient, result: NewEngineResult) {
  const [{ maxVersion }] = await db
    .select({ maxVersion: sql`COALESCE(MAX(version), 0)` })
    .from(engineResults)
    .where(and(
```

```
      eq(engineResults.dealId, result.dealId),
      eq(engineResults.engineName, result.engineName)
    ));
  return db.insert(engineResults).values({
    ...result,
    version: maxVersion + 1,
  }).returning();
}
```

Query latest: Use DISTINCT ON or a subquery to get the most recent version per engine.

```
export async function getLatestEngineResult(
  db: DrizzleClient, dealId: string, engineName: string
) {
  return db.select()
    .from(engineResults)
    .where(and(
      eq(engineResults.dealId, dealId),
      eq(engineResults.engineName, engineName)
    ))
    .orderBy(desc(engineResults.version))
    .limit(1);
}
```

5.4 Canonical Queries for MVP Screens

Screen	Query Function	Tables Joined	Key Logic
Dashboard	getDealDashboard(dealId)	deals + latest of each engine_result + latest recommendation + recent audit_log	Single deal fetch + 5 "get latest" subqueries + last 20 audit entries. Returns DealDashboardView.
Scenario Explorer	getScenarioExplorer(dealId)	deals + engine_results WHERE engine IN (underwriter, montecarlo, factor)	Latest underwriter result per scenario (bear/base/bull) + latest MC + latest factor. Pivots into comparison table server-side.
Budget Tracker	getBudgetTracker(dealId, month)	deals + budget_lines + rfis + change_orders + engine_results(budget, scurve)	Aggregate budget_lines up to month. Join open RFIs and pending COs. Latest budget engine + scurve for chart data.

Dashboard Query (sketch)

```
export async function getDealDashboard(db: DrizzleClient, dealId: string)
  : Promise<DealDashboardView> {
  const deal = await db.select().from(deals).where(eq(deals.id, dealId)).limit(1);
  const [latestFactor, latestPf, latestMC, latestBudget, latestRec] = await Promise.all([
    getLatestEngineResult(db, dealId, "factor"),
    getLatestEngineResult(db, dealId, "underwriter"),
    getLatestEngineResult(db, dealId, "montecarlo"),
    getLatestEngineResult(db, dealId, "budget"),
    db.select().from(recommendations)
      .where(eq(recommendations.dealId, dealId))
      .orderBy(desc(recommendations.version)).limit(1),
  ]);
  const recentEvents = await db.select().from(auditLog)
    .where(eq(auditLog.dealId, dealId))
    .orderBy(desc(auditLog.timestamp)).limit(20);
  const recHistory = await db.select().from(recommendations)
    .where(eq(recommendations.dealId, dealId))
    .orderBy(desc(recommendations.version)).limit(10);
```

```
  return assemblesDashboardView(deal, latestFactor, latestPf, latestMC,
    latestBudget, latestRec, recentEvents, recHistory);
}
```

5.5 Migration Strategy

- **Tool:** drizzle-kit for schema diffing and SQL generation. Review generated SQL before applying.
- **Naming:** Sequential numbered files: 0001_initial.sql, 0002_add_risk_register.sql, etc.
- **Environments:** Dev uses Docker Compose Postgres. Staging/prod use managed Postgres (Neon or RDS). Migrations run in CI before deployment.
- **Seed:** v3grand.ts inserts the full V3 Grand deal with all assumptions from the Excel sheets. This is the golden test fixture.

6. UI Contracts & Wireframes (packages/ui)

6.1 File Layout

```
packages/ui/
  src/
    app/
      layout.tsx          # root layout: auth provider, query client, nav shell
      page.tsx            # deal list (redirect to single deal for MVP)
      deals/[dealId]/
        layout.tsx        # deal-level nav tabs
        page.tsx          # Deal Dashboard screen
        scenarios/page.tsx # Scenario Explorer screen
        budget/page.tsx   # Budget Tracker screen
    components/
      dashboard/          # components for Deal Dashboard
      scenarios/          # components for Scenario Explorer
      budget/             # components for Budget Tracker
      shared/             # RecommendationBadge, MetricCard, etc.
    hooks/
      use-deal.ts         # useDeal(dealId) → Deal
      use-dashboard.ts    # useDashboard(dealId) → DealDashboardView
      use-scenarios.ts    # useScenarios(dealId) → ScenarioExplorerView
      use-budget.ts       # useBudget(dealId, month) → BudgetTrackerView
      use-engine.ts       # useRunEngine(dealId, engine) → mutation hook
      use-websocket.ts    # WebSocket subscription for real-time events
    lib/
      api-client.ts       # typed fetch wrapper over Fastify API
      query-keys.ts       # centralized React Query key factory
```

6.2 React Query Hooks

Every hook wraps a TanStack Query call. The API client is typed using the view DTOs from @v3grand/core.

```
// hooks/use-dashboard.ts
import { useQuery } from "@tanstack/react-query";
import { DealDashboardView } from "@v3grand/core";
import { apiClient } from "../lib/api-client";
import { queryKeys } from "../lib/query-keys";

export function useDashboard(dealId: string) {
  return useQuery<DealDashboardView>({
    queryKey: queryKeys.dashboard(dealId),
    queryFn: () => apiClient.get(`/deals/${dealId}/dashboard`),
    staleTime: 30_000,  // 30s before refetch
    refetchOnWindowFocus: true,
  });
}
// hooks/use-engine.ts (mutation)
import { useMutation, useQueryClient } from "@tanstack/react-query";

export function useRunEngine(dealId: string, engine: string) {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: () => apiClient.post(`/deals/${dealId}/engines/${engine}`),
    onSuccess: () => {
      // Invalidate affected views so they refetch with new engine results
      qc.invalidateQueries({ queryKey: queryKeys.dashboard(dealId) });
      qc.invalidateQueries({ queryKey: queryKeys.scenarios(dealId) });
```

```
    },
  });
}
// lib/query-keys.ts
export const queryKeys = {
  deals: () => ["deals"] as const,
  deal: (id: string) => ["deals", id] as const,
  dashboard: (id: string) => ["deals", id, "dashboard"] as const,
  scenarios: (id: string) => ["deals", id, "scenarios"] as const,
  budget: (id: string, month?: number) => ["deals", id, "budget", month] as const,
  engineHistory: (id: string, engine: string) =>
    ["deals", id, "engines", engine, "history"] as const,
  recommendations: (id: string) => ["deals", id, "recommendations"] as const,
};
```

6.3 Screen 1: Deal Dashboard

Route: /deals/[dealId]

Hook: useDashboard(dealId) → DealDashboardView

Purpose: Single-screen overview of the entire deal: current recommendation, key financials, construction status, and recent activity.

Wireframe (text layout)

```
┌──────────────────────────────────────────────────────────────┐
│  DEAL HEADER                                                 │
│  V3 Grand Madurai  |  Phase: Construction  |  Month 14       │
├──────────────────────┬──────────────────────┬────────────────┤
│ RECOMMENDATION       │ KEY METRICS          │ CONSTRUCTION   │
│                      │                      │                │
│ [INVEST]             │ IRR: 15.7%           │ 64% complete   │
│ Confidence: 69       │ NPV: 4.2 Cr          │ 12 days ahead  │
│ Factor: 3.7/5.0      │ Eq Mult: 2.1x        │ 3 open RFIs    │
│ Since: v4 (Mar 26)   │ DSCR: 1.45x          │ 1 pending CO   │
├──────────────────────┴──────────────────────┴────────────────┤
│ RECOMMENDATION HISTORY (sparkline timeline)                  │
│ v1[INVEST] → v2[INVEST] → v3[HOLD] → v4[INVEST]              │
├──────────────────────────────┬───────────────────────────────┤
│ BUDGET SNAPSHOT              │ RECENT ACTIVITY               │
│ Total: 35.7 Cr               │ Mar 3 - Assumption updated    │
│ Spent: 22.8 Cr (64%)         │ Mar 2 - Engine revaluation    │
│ Forecast: 36.1 Cr (+1.1%)    │ Mar 1 - RFI #12 resolved      │
│ [mini S-curve chart]         │ Feb 28 - CO #3 approved       │
└──────────────────────────────┴───────────────────────────────┘
```

Components

Component	Props (from DealDashboardView)	Renders
<DealHeader />	deal, property	Name, phase badge, month count, location, star rating
<RecommendationCard />	latestRecommendation	Verdict badge (color-coded), confidence bar, factor score, version history link
<KeyMetricsCard />	latestProforma, latestMC	IRR, NPV, equity multiple, DSCR, MC P50 IRR. Each as a labeled value.
<ConstructionCard />	constructionProgress	Percent complete bar, days ahead/behind, open RFI count, pending CO count
<RecommendationTimeline />	recommendationHistory	Horizontal timeline with verdict badges. Click to expand explanation.

Component	Props (from DealDashboardView)	Renders
<BudgetSnapshotCard />	budgetSummary	Budget/spent/forecast summary, mini S-curve (Recharts AreaChart), variance %
<ActivityFeed />	recentEvents	Scrollable list of recent events with actor, action, timestamp

6.4 Screen 2: Scenario Explorer

Route: /deals/[dealId]/scenarios

Hook: useScenarios(dealId) → ScenarioExplorerView

Wireframe

```
┌──────────────────────────────────────────────────────────────────┐
│ SCENARIO COMPARISON TABLE                                         │
│ ┌──────────────┬──────────┬──────────┬──────────┐                 │
│ │ Metric       │ Bear     │ Base     │ Bull     │                 │
│ ├──────────────┼──────────┼──────────┼──────────┤                 │
│ │ IRR          │ 10.2%    │ 15.7%    │ 22.1%    │                 │
│ │ NPV (Cr)     │ -1.2     │ 4.2      │ 9.8      │                 │
│ │ Equity Mult  │ 1.5x     │ 2.1x     │ 2.8x     │                 │
│ │ DSCR         │ 1.1x     │ 1.45x    │ 1.8x     │                 │
│ │ Occ (stab)   │ 65%      │ 78%      │ 85%      │                 │
│ │ ADR (stab)   │ 4,200    │ 5,000    │ 5,800    │                 │
│ └──────────────┴──────────┴──────────┴──────────┘                 │
├──────────────────────────────┬──────────────────────────────────┤
│ MC IRR HISTOGRAM             │ FACTOR SCORE RADAR                 │
│ [Recharts BarChart]          │ [Recharts RadarChart]              │
│ P10 | P50 | P90 markers      │ Global / Local / Asset / Spon.     │
│ P(NPV<0) = 8.7%              │ Composite: 3.7 / 5.0               │
├──────────────────────────────┴──────────────────────────────────┤
│ 10-YEAR CASH FLOW CHART (all 3 scenarios overlaid)                 │
│ [Recharts LineChart]  Y-axis: FCFE (Cr)  X-axis: Year 1..10        │
└──────────────────────────────────────────────────────────────────┘
```

Components

Component	Props	Renders
<ScenarioTable />	comparisonTable[]	Pivoted metric × scenario table. Conditional coloring (red if IRR < WACC).
<MCHistogram />	mcSummary.histogram, irrDistribution	Bar chart of IRR buckets. Vertical lines at P10/P50/P90. Shaded region below 0.
<FactorRadar />	factorSummary.domainScores	Radar/spider chart: 4 axes (Global, Local, Asset, Sponsor). Center = 1, edge = 5.
<CashFlowChart />	scenarios[].proforma.cashFlows	3-line overlay chart. X = year, Y = FCFE. Legend: Bear (red), Base (blue), Bull (green).
<RunEngineButton />	dealId, engine name	Triggers useRunEngine mutation. Shows loading spinner. Disabled during runs.

6.5 Screen 3: Budget Tracker

Route: /deals/[dealId]/budget

Hook: useBudget(dealId, selectedMonth) → BudgetTrackerView

Wireframe

```
┌──────────────────────────────────────────────────────────────┐
│ BUDGET SUMMARY CARDS                                          │
│ [Total: 35.7 Cr] [Spent: 22.8 Cr] [Forecast: 36.1 Cr]        │
│ [Variance: +1.1%] [Committed: 30.2 Cr]                        │
├──────────────────────────────────────────────────────────────┤
│ S-CURVE CHART                                                 │
│ [Recharts AreaChart]                                          │
│ 3 lines: Planned (dashed), Actual (solid), Forecast (dotted)  │
│ X-axis: months 0..36   Y-axis: cumulative INR (Cr)            │
├──────────────────────────────────────────────────────────────┤
│ TOP VARIANCES TABLE                                           │
│ Cost Code | Description | Budget | Forecast | Var | Var%      │
│ STRUC-001 | Structural  | 8.2 Cr | 8.5 Cr  |+0.3 | +3.7%     │
│ MEP-003   | HVAC        | 3.1 Cr | 3.4 Cr  |+0.3 | +9.7%     │
├──────────────────────────────┬───────────────────────────────┤
│ RFIs (3 open)                │ CHANGE ORDERS (1 pending)     │
│ #12 - HVAC spec [HIGH]       │ #3 - Foundation upgrade       │
│ #11 - Tile spec  [MED]       │     +2.1 Cr | Awaiting approval│
│ #10 - MEP coord  [LOW]       │                               │
├──────────────────────────────┴───────────────────────────────┤
│ RISK EXPOSURE BY CATEGORY                                     │
│ [Recharts horizontal BarChart]                                │
│ Construction: 3 risks, 1.2 Cr exposure                        │
│ Market: 2 risks, 0.8 Cr exposure                              │
└──────────────────────────────────────────────────────────────┘
```

Components

Component	Props	Renders
<BudgetSummaryCards />	summary (totals)	Five metric cards: total, spent, forecast, committed, variance. Color-coded.
<SCurveChart />	sCurveChart[]	Recharts AreaChart. Planned=dashed gray, Actual=solid blue, Forecast=dotted orange.
<VarianceTable />	topVariances[]	Sortable table. Rows highlighted red if variance > 10%, amber if > 5%.
<RFIList />	rfis[]	Compact list with priority badges, status pills, cost/schedule impact.
<ChangeOrderList />	changeOrders[]	List with status flow (draft > submitted > under-review > approved). Approve button for lead-investor.
<RiskExposureChart />	riskExposure[]	Horizontal bar chart grouped by category. Bar length = total exposure.

6.6 WebSocket Integration

```
// hooks/use-websocket.ts
export function useDealWebSocket(dealId: string) {
  const qc = useQueryClient();
  useEffect(() => {
    const ws = new WebSocket(`${WS_URL}/deals/${dealId}/events`);
    ws.onmessage = (e) => {
      const event: EventEnvelope = JSON.parse(e.data);
      // Invalidate relevant queries based on event type
      if (event.payload.type === "engine.completed") {
        qc.invalidateQueries({ queryKey: queryKeys.dashboard(dealId) });
        qc.invalidateQueries({ queryKey: queryKeys.scenarios(dealId) });
      }
```

```
      if (event.payload.type === "budget.actual.updated") {
        qc.invalidateQueries({ queryKey: queryKeys.budget(dealId) });
      }
      if (event.payload.type === "recommendation.changed") {
        qc.invalidateQueries({ queryKey: queryKeys.dashboard(dealId) });
        // Show toast notification for recommendation flips
        toast.info(`Recommendation updated to ${event.payload.to}`);
      }
    };
    return () => ws.close();
  }, [dealId]);
}
```

7. Developer Onboarding Checklist

7.1 Required Reading (Day 1)

Order	Document	Sections to Focus On	Time
1	V3-Grand-Platform-Blueprint-v2.docx	Sections 2 (Domain Model), 3 (Events), 4 (Service Architecture). Skim sections 5-7.	90 min
2	V3-Grand-Implementation-Plan.docx	Section 1 (MVP Scope — what is in/out). Section 2 (Tech Stack). Section 7 (Delivery Plan).	60 min
3	This document (Developer Scaffolding Guide)	Section 2 (types) and Section 3 (engines) are the code you will write first. Section 1 for repo setup.	90 min
4	Excel data files (in repo /data/)	Key Assumptions, CAPEX & Phasing, EBITDA & CF sheets. These are the source-of-truth numbers your engines must reproduce.	30 min

7.2 Environment Setup (Day 1)

Follow these steps in order. Each step depends on the previous one.

Step 1: Prerequisites

- Node.js 20 LTS, pnpm 9+, Docker Desktop (for Postgres, Redis, NATS, Temporal)
- GitHub access to the v3grand monorepo
- Clerk dev account (or Auth0 sandbox) for auth tokens

Step 2: Clone and install

```
git clone git@github.com:v3grand/v3grand.git
cd v3grand
pnpm install        # installs all workspaces
pnpm build          # builds core first (others depend on it)
```

Step 3: Start infrastructure

```
cd packages/infra
docker compose up -d    # Postgres, Redis, NATS, Temporal server
# Verify:
docker compose ps       # all services should be "running"
# Temporal UI at http://localhost:8233
```

Step 4: Database

```
pnpm db:migrate         # runs Drizzle migrations against local Postgres
pnpm db:seed            # inserts V3 Grand deal with all assumptions
```

Step 5: Start API server

```
pnpm --filter @v3grand/api dev
# Verify: curl http://localhost:3001/deals  (should return seeded deal)
```

Step 6: Start Temporal worker

```
pnpm --filter @v3grand/workflows dev
# Verify: check Temporal UI shows "v3grand-main" task queue
```

Step 7: Start UI

```
pnpm --filter @v3grand/ui dev
# Open http://localhost:3000  (should show Deal Dashboard for seeded deal)
```

7.3 Week 1: Walking Skeleton

The goal for Week 1 is a single end-to-end loop: create a deal, run an engine, see a result on the dashboard. This validates every layer of the stack.

Walking Skeleton Scope

Day	Task	Package	Definition of Done
Mon	Monorepo init + core types	core	pnpm build succeeds. All entity types compile. Zod validators pass for V3 Grand fixture.
Mon	DB schema + seed	db	Migrations run. Seed inserts deal. SELECT * FROM deals returns 1 row with valid JSONB.
Tue	Underwriter engine (base scenario)	engines	buildProForma(v3grandDeal, "base") returns IRR ~15.7%, NPV > 0. Test passes.
Tue	Factor engine (stub)	engines	scoreFactors(v3grandDeal) returns composite ~3.7. Hardcoded factor scores OK for skeleton.
Wed	API: GET /deals/:id, POST .../engines/underwriter	api	curl returns seeded deal. POST triggers engine, persists result, returns ProFormaOutput.
Wed	API: GET /deals/:id/dashboard	api	Returns DealDashboardView with deal + latest engine results (even if partial).
Thu	Decision engine (stub)	engines	evaluate() with Factor + Underwriter + stub MC returns INVEST/HOLD. Confidence > 0.
Thu	Recommendation persistence	api + db	POST /deals/:id/engines/decision creates row in recommendations table. GET /dashboard includes it.
Fri	UI: Dashboard shell	ui	Next.js page renders. useDashboard() fetches from API. Shows DealHeader, RecommendationCard, KeyMetricsCard with real data.
Fri	Run full loop manually	all	In browser: see deal → click "Run Underwriter" → click "Run Decision" → recommendation updates on dashboard.

7.4 Key Architectural Invariants to Internalize

Before writing any code, make sure these principles are second nature:

- **Engines are pure.** They take a plain object, return a plain object. No database, no HTTP, no file system. Test them with vitest in isolation.
- **Versioned results are append-only.** Never UPDATE engine_results or recommendations. Always INSERT a new row with version = previous + 1.
- **Events drive cascades.** When an assumption changes, you do not manually call every engine. You emit assumption.updated, which triggers a Temporal workflow that orchestrates the engine chain.
- **The UI is a thin read layer.** It calls pre-assembled view endpoints (DealDashboardView, etc.). It never calls engines directly or joins data client-side.
- **RBAC is at the API gateway.** Engines do not know about roles. Routes enforce who can call what.
- **Types flow from core.** If you need to change a type, change it in @v3grand/core. The TypeScript compiler will show you every downstream consumer that needs updating.

7.5 Common Pitfalls

Pitfall	Why It Happens	How to Avoid
Importing engines in UI	Tempting to run a quick calc client-side	ESLint boundary rule blocks it. All engine calls go through the API.
Mutating the deal object in an engine	MC temporarily injects a _mc scenario	Deep-clone the deal before mutation, or use a functional approach.
Forgetting to version engine results	Updating the "latest" row instead of inserting	insertEngineResult() auto-increments version. Never write a raw UPDATE.
Hardcoding V3 Grand values	Easy to bake in 55 keys, 35.7 Cr, etc.	All values come from the Deal object passed to engines. Engines are deal-agnostic.
Running Temporal workflows synchronously in API routes	Waiting for full engine cascade in a single HTTP request	Start the workflow, return 202 Accepted. UI polls or uses WebSocket for completion.

7.6 Reference Links

- **Temporal TypeScript SDK:** docs.temporal.io/typescript
- **Drizzle ORM:** orm.drizzle.team/docs
- **Fastify:** fastify.dev/docs
- **TanStack React Query:** tanstack.com/query
- **NATS JetStream:** docs.nats.io/jetstream
- **Recharts:** recharts.org/api
- **Zod:** zod.dev